

Défrichage

Chat Gpt entrées :

- Je suis un étudiant en première année de licence et je découvre le C. Je veux que tu me fasses un petit cours sur les fonctions en C. Jusque là je n'ai vu que les variables, les conditions et les boucles. Je ne souhaite pas voir les passages par références, ni les pointeurs. Je souhaite en revanche avoir une vue d'ensemble des fonctions et des détails sur les liens avec la portée des variables et ce que signifie passage par valeur ou copie.
- Quelles sont les différences entre les fonctions en python et en C ?

Qu'est-ce qu'une fonction / méthode ?



Une boîte qui contient un algorithme :

- Des entrées
- Un traitement (du code)
- Une sortie (pour le moment)

Pour quoi faire ?

- Ne pas répéter
- Faciliter la maj
- Rendre plus lisible le code
- Ranger les fichiers / bibliothèques

Déclarer une fonction

Type de
l'information en
sortie

Nom de la
fonction

Les entrées : nom de
variables avec leur type.
Sont aussi appelés : arguments
ou paramètres.

```
int myFunction ( int entry1, double arg2) //prototype de la fonction
{
    //le traitement : le corps de la fonction
}
```

Déclarer une fonction : le nom

```
int myFunction( int entry1, double arg2) //prototype de la fonction
{
    //le traitement : le corps de la fonction
}
```

Conventions pour nommer une fonction :

- En anglais
- Commence par un verbe
- Démarre par une lettre de l'alphabet, en minuscule
- Chaque autre mot démarre par une majuscule
- Des numéros sont possibles (mais pas en premier caractère)
- Pas de symboles spéciaux

Portée des variables & fonctions

La portée : où une variable peut-elle être appelée dans le code ?

Portée d'une variable en C/C++ : un bloc `{ }` (et ceux imbriqués).

Le *main* est un bloc. Une fonction en est un autre.

```
int myFunction ( int entry1, double arg2) //prototype de la fonction
{
    //le traitement : le corps de la fonction
}
```

Les variables dans le *main* sont invisibles dans la fonction et vice-versa.

Portée des variables et paramètres

Les paramètres (ou arguments) sont le moyen de faire passer des valeurs à la fonction. Des variables sont créées et les valeurs y sont stockées.

```
int myFunction( int entry1, double arg2 ) //prototype de la fonction
{
    //le traitement : le corps de la fonction
}
```

Appel de la fonction :

```
myfunction ( 3, 5.4 );
```

Paramètres : passage par copie

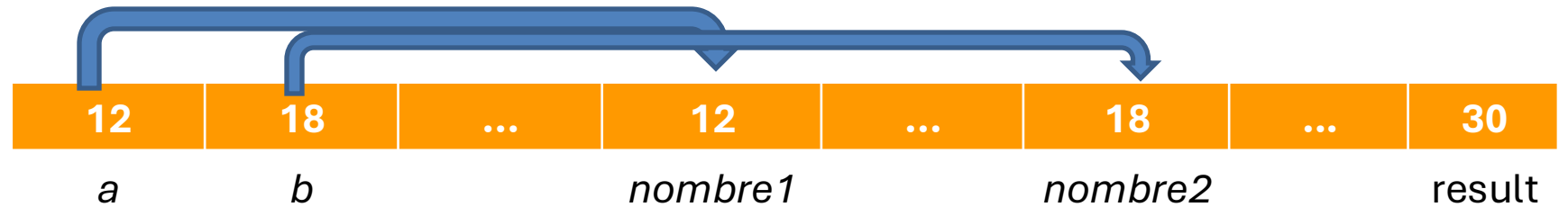
ATTENTION : En python, le passage par défaut est le passage par référence

```
void sum ( int number1, int number2 )  
{  
    int result;  
    result = number1 + number2;  
    printf("%d\n", result);  
}
```

```
int main()  
{  
    int a;  
    int b;  
    scanf("%d",&a);  
    scanf("%d",&b);  
  
    sum(a,b);  
}
```

On envoie les valeurs de a et b

Mémoire
Variables



Paramètres : passage par copie

```
void addOne( int a )  
{  
    a = a + 1;  
}
```

```
int main()  
{  
    int a;  
    a = 15;  
  
    addOne(a);  
    printf("%d\n",a);  
}
```

Mémoire
Variables



Paramètres : passage par copie

```
void addOne( int a )
{
    a = a + 1;
}

int main()
{
    int a;
    a = 15;

    addOne(a);
    printf("%d\n",a);
}
```

```
void addOne( int a )
{
    a = a + 1;
}

int main()
{
    addOne(15);
    printf("15\n");
}
```

```
void addOne( int number )
{
    number = number + 1;
}

int main()
{
    int a;
    a = 15;

    addOne(a);
    printf("%d\n",a);
}
```

Mémoire

Variables

15

...

...

...

16

...

...

...

*a**number*

La pile et les fonctions

Rappel

Zone dans laquelle la variable existe, l'état de la pile actuelle.

Une variable existe dans :

- le bloc dans lequel elle est déclarée
- les blocs fils (bloc imbriqué).

Dès que l'on sort du bloc elle n'existe plus.

	main() : w	3
main : if : v --> [5]	main() : if : z v	3
	a() : x	1
	b() : y	2

```
void c(){
    int z = 3;
}
```

```
void b(){
    int y = 2;
}
```

```
void a() {
    int x = 1;
    b();
}
```

```
int main(){
    int w = 3;
    if (w == 3)
    {
        int v = 5;
        a();
    }
    c();
}
```

Pile, portée des variables et retour du résultat

```
void addOne( int number )  
{  
    number = number + 1;  
}
```

```
int main()  
{  
    int a;  
    a = 15;  
  
    addOne(a);  
    printf("%d %d\n", a, number);  
}
```

erreur

Mémoire
Variables



a

~~number~~

Quand on sort de la fonction

Portée des variables et retour du résultat par copie

Comment la fonction peut-elle alors renvoyer **un** résultat ?

Type attendu
(si aucun retour
attendu alors le
type est **void**)

```
int myFunction ( int entry1, double arg2) //prototype de la fonction
{
    //le traitement : le corps de la fonction
}
```

```
void addOne( int number )
{
    number = number + 1;
}

int main()
{
    int a;
    a = 15;

    addOne(a);
    printf("%d %d\n",a, number);
}
```



```
int addOne( int number )
{
    number = number + 1;
    return number;
}

int main()
{
    int a;
    a = 15;

    int result;
    result = addOne(a);
    printf("%d %d\n",a, result);
}
```

return quitte la fonction

```
bool isAboveTen ( int number )  
{  
    if ( number > 10 )  
    {  
        return true;  
    }  
    else  
    {  
        return false;  
    }  
}
```

=

```
bool isAboveTen ( int number )  
{  
    if ( number > 10 )  
    {  
        return true;  
    }  
    return false;  
}
```

Où mettre le code ? : créer des "bibliothèques"

```
#include <stdio.h>

int main()
{
    printf("Hello world\n");
}
```

Inclusion de la
bibliothèque *stdio* :

- *printf*
- *scanf*
- ...

Fichier .h

```
#ifndef MATHOPERATION_H
#define MATHOPERATION_H

int addOne ( int a );
double sum ( double a, double b );
int power ( int a, int pow );

#endif
```

mathematicOperations.h

```
#include <stdio.h>
#include "mathematicOperations.h"

int main()
{
    int result;
    result = addOne(12);
    printf("Résultat : %d\n", result);
}
```

main.c

Fichier .c

```
#include "mathematicOperations.h"

int addOne ( int a )
{
    a = a +1;
    return a;
}

double sum (double a, double b)
{
    double sum;
    sum = a + b;
    return sum;
}
```

mathematicOperations.c

Surcharge

Deux fonctions peuvent s'appeler pareil si la liste des arguments est différente.

Permet des traitements différents selon la qualité et quantité des paramètres (et non leur valeur).

Il doit y avoir des modifications ici : ajout ou suppression de variables et/ou changement des types. Les noms ne compte pas.

Le type de retour peut changer ou rester le même

```
int myFunction ( int entry1, double arg2) //prototype de la fonction
{
    //le traitement : le corps de la fonction
}
```

Exemples:

```
int maFonction ( int entree1 )
double maFonction ( int entree1 )
× double maFonction ( int toto, double arg2 )
void maFonction ( )
void maFonction( int entree1, int arg2)
bool maFonction ( int entree1, double arg2, bool param3)
```